

DeltaFrame: Design-Change Memory for AI Coding Agents

A Figma plugin, backend, MCP server, and Codex skill that help designers turn section iterations into clear, designer-level changelogs for humans and AI implementation tools.

Stage Private beta prototype	Audience Designers + AI coders	Positioning Complements Figma MCP
--	--	---

Core thesis: Figma MCP can tell an AI what a design looks like now. DeltaFrame answers the narrower handoff question: what changed since the last implementation baseline?

1. Problem

Product designers iterate quickly, often by duplicating a section or frame and exploring a new version beside the old one. Developers and AI coding agents do not always need the entire design context again; they need a concise, trustworthy explanation of what changed.

Current pain	Opportunity
• Design changes are buried in duplicated frames and renamed layers. • Handoff notes are manual, inconsistent, or missing. • AI coding tools can over-read the whole design instead of updating only affected code. • Small but important changes such as spacing, copy, variants, and states are easy to miss.	• Capture design iteration history directly from Figma. • Generate designer-readable changelogs on the canvas. • Expose compact approved context through MCP. • Help AI agents update only what changed.

2. Product Concept

DeltaFrame tracks section-based design iteration. A designer selects a Figma section, saves it as a baseline, copies it for iteration, and then asks DeltaFrame to create a changelog. The plugin compares the selected iteration with the previous tracked version and writes a visible changelog frame beside the design.

Simple user promise: Select a section, click Create changelog, and get a useful designer-level summary of what changed.
--

Change language: moved, resized, padding changed, spacing changed, color changed, font changed, copy changed, added, removed, hidden, variant changed.

3. Current Architecture

Figma Plugin	Backend
• Captures selected section snapshots using Figma JSON_REST_V1. • Exports PNG screenshots for visual enhancement. • Stores section and iteration ids in shared plugin data. • Creates or updates visible changelog frames beside iterations. • Centers around one action: Create changelog.	• Node + Hono API with file-backed SQLite via sql.js. • Stores projects, sections, iterations, checkpoints, snapshots, and changelogs. • Performs deterministic semantic matching, deep diffs, designer-level grouping, and optional Gemini enhancement. • Provides smart section matching for untagged copied iterations.

MCP Server • Exposes section-aware tools for AI coding agents. • Provides latest section delta and compact implementation briefs. • Lets agents mark iterations implemented after code changes. • Delivers compact approved or draft context; backend owns storage and diffing.	Codex Skill • Teaches Codex how to consume DeltaFrame MCP tools. • Prioritizes section timelines and latest deltas. • Uses Figma MCP only when the DeltaFrame brief is ambiguous or low confidence. • Encourages targeted code changes instead of full-screen reimplementation.
--	--

4. Simplified Workflow

- 1 **Select a Figma section.** If it is new, DeltaFrame treats it as an untracked section.
- 2 **Click Create changelog.** The plugin exports the snapshot internally; there is no visible export step.
- 3 **First click creates a baseline.** The selected node is tagged with a section id and baseline iteration id.
- 4 **Copy and change the section.** The designer works naturally on a duplicated section/frame.
- 5 **Click Create changelog again.** DeltaFrame smart-matches untagged copies or uses existing metadata.
- 6 **Draft changelog appears on canvas.** The frame summarizes designer-level changes and includes the MCP changelog id.
- 7 **Optional review and approval.** Designers can edit notes and approve the changelog for MCP consumption.

5. Comparison Logic

DeltaFrame keeps deterministic data as the source of truth. It normalizes Figma exports into stable trees and compares iterations through semantic node matching, raw property diffs, and designer-level aggregation.

Semantic matching	Designer-level aggregation
• Matches by Figma node id when possible. • Falls back to type, name similarity, text, component keys, main component ids, size, position, depth, and parent role. • Stores match confidence and match reason for review. • Ignores root-frame placement offsets caused by copying a section beside the baseline.	• Groups raw x/y changes into one moved event. • Groups width/height into resized. • Groups padding and gap changes into spacing notes. • Groups typography, color, component variant, token, copy, visibility, added, and removed changes.

6. Gemini Enhancement

Gemini is optional and is not required for correctness. The backend first generates a deterministic changelog from Figma JSON. If a Gemini key is configured, screenshots and deterministic diffs are sent to Gemini to improve summary quality, grouping, implementation notes, and obvious visual-only observations.

Design principle: AI can improve language and catch visible context, but it should not replace the deterministic evidence used to decide what changed.

7. MCP and AI Agent Use

DeltaFrame MCP exposes section-aware tools such as `list_sections`, `get_section_timeline`, `get_latest_section_delta`, `get_section_agent_brief`, and `mark_iteration_implemented`.

- 1 Find the relevant tracked section.
- 2 Fetch the latest approved or draft section delta.
- 3 Read the compact agent brief.
- 4 Use Figma MCP only if exact current design context is needed.
- 5 Update only the affected code.
- 6 Mark the iteration implemented.

8. Current State

- TypeScript monorepo scaffold exists for plugin, backend, MCP server, and core package.
- Section and iteration timelines are implemented.
- Designer-level changelog grouping is implemented.
- One-click plugin flow is implemented in the generated plugin UI.
- Backend smart-match endpoint exists for untagged copied iterations.

- Gemini enhancement is optional and falls back to deterministic changelogs.
- Automated checks cover core diffing, designer grouping, section timeline storage, and smart matching.

9. Open Questions for Feedback

Product • Is "design-change memory for AI coding agents" the clearest positioning? • Should the primary user be designers, developers, or design-system teams? • Should canvas changelog frames be the main output, or should the plugin review UI be primary? • What would make teams trust generated design changelogs?	Workflow • Is one-click baseline/changelog creation simple enough? • When should a changelog be auto-approved for MCP, if ever? • How much control should advanced users have over matching, thresholds, and grouping? • Should designers be able to compare against any prior iteration, not only the previous one?
Technology • How robust should visual diffing become beyond Figma JSON metadata? • Should Gemini or another LLM produce higher-level product implications? • What is the right storage model for teams: local, hosted SaaS, or both? • How should this integrate with existing Figma MCP workflows?	Go-to-market • Who feels this pain most acutely: designers using AI coders, developers consuming Figma, or design systems teams? • Would people install a Figma plugin for this, or expect it inside their coding agent? • What would be a compelling private beta test? • What demo file or workflow would best prove the value?

10. Short Pitch

DeltaFrame helps designers and AI coding agents understand what changed between Figma design iterations. It turns copied sections into tracked timelines, writes readable changelog frames on the Figma canvas, and exposes compact implementation briefs through MCP so AI coding tools can update only the affected code.

Prepared as a portable project synopsis for feedback, critique, and iteration in other AI tools.